

Multimodal Visual Understanding + SLAM Navigation

Multimodal Visual Understanding + SLAM Navigation

1. Course Content
2. Start the Agent
 - 2.1 Configure Map Mapping File
 - 2.2 Configuring Map Mapping Variables in Dify
3. Running the Case
 - 3.1 Startup Program
 - 3.2 Test Case

1. Course Content

- Run the example program to perform a comprehensive task using the robot's visual understanding capabilities combined with SLAM navigation through text interaction.

2. Start the Agent

Note: If it's already started, you don't need to start it again.

Enter the command on the vehicle terminal:

```
start_agent
```

The terminal will print the following information, indicating a successful connection:

```
mircoROS_Agent
subscriber created | client_key: 0x1D229138, subscriber_id: 0x001(4), participant_id: 0x000(1)
[1767693398.286449] info | ProxyClient.cpp | create_datareader | datareader created | client_key: 0x1D229138, datareader_id: 0x001(6), subscriber_id: 0x001(4)
[1767693398.290468] info | ProxyClient.cpp | create_topic | topic created | client_key: 0x1D229138, topic_id: 0x007(2), participant_id: 0x000(1)
[1767693398.294835] info | ProxyClient.cpp | create_subscriber | subscriber created | client_key: 0x1D229138, subscriber_id: 0x002(4), participant_id: 0x000(1)
[1767693398.299603] info | ProxyClient.cpp | create_datareader | datareader created | client_key: 0x1D229138, datareader_id: 0x002(6), subscriber_id: 0x002(4)
[1767693398.303485] info | ProxyClient.cpp | create_topic | topic created | client_key: 0x1D229138, topic_id: 0x008(2), participant_id: 0x000(1)
[1767693398.330727] info | ProxyClient.cpp | create_subscriber | subscriber created | client_key: 0x1D229138, subscriber_id: 0x003(4), participant_id: 0x000(1)
[1767693398.335405] info | ProxyClient.cpp | create_datareader | datareader created | client_key: 0x1D229138, datareader_id: 0x003(6), subscriber_id: 0x003(4)
```

[!NOTE]

Note: To experience this lesson, you need to build at least one grid map according to the LiDAR chapter.

2.1 Configure Map Mapping File

- Command to Start Navigation on the Vehicle Terminal

For Raspberry Pi 5 users, if the ROS container is not started, you need to start it first.

```
bringup_m3
```

Open the terminal inside the container.

```
exec_m3
```

(ORIN mainboard)

```
ros2 launch m3_bringup nav2.launch.py
```

(RDK & PI5 mainboard) Step-by-step execution, visualization is performed on the accompanying virtual machine `Rosmaster-M3-Ubuntu22.04`.

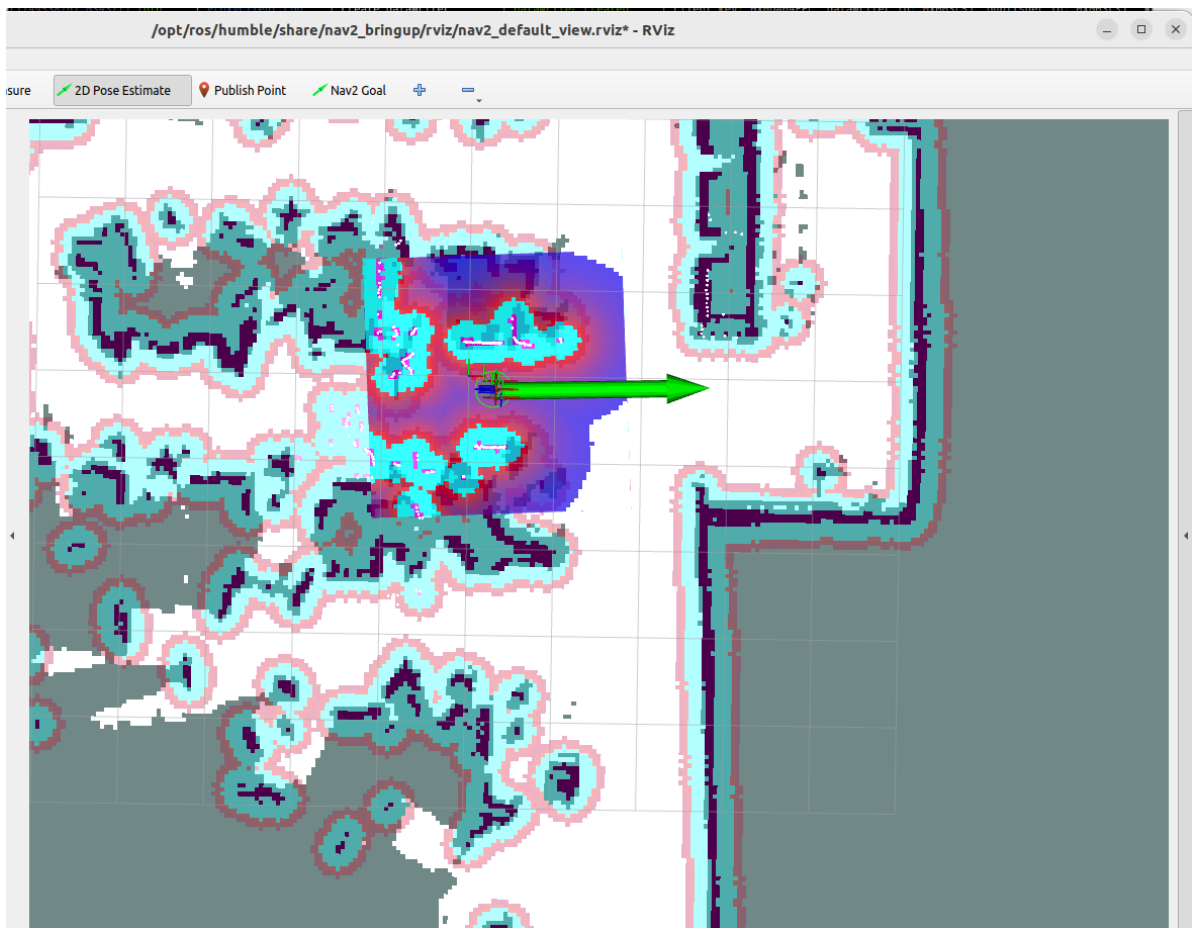
```
#Virtual machine side
rviz2 -d /home/yahboom/yahboomM3_ws/user_ws/src/m3_bringup/rviz/nav2.rviz
#Host side
ros2 launch m3_bringup nav2.launch.py gui:=False
```

[!TIP]

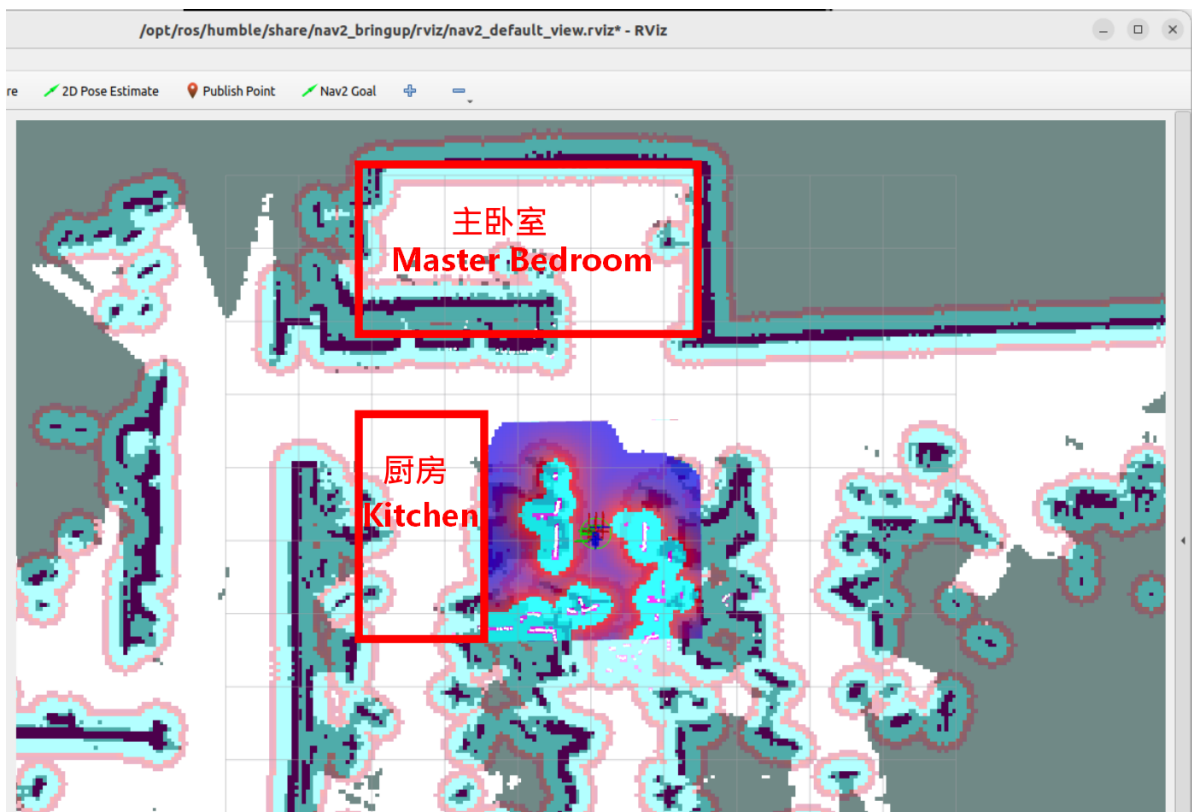
- This uses the AMCL positioning algorithm for navigation by default, suitable for most applicable scenarios. If higher positioning accuracy is required, refer to the product tutorial [02.Lidar Course]—[13.slam_toolbox localization and navigation] section to build a pose map and use the slam_toolbox pure positioning mode for navigation.

Next, the rviz2 visualization interface will open. Click on **2D Pose Estimate** in the top toolbar to enter selection mode. Roughly mark the robot's location and orientation on the map.

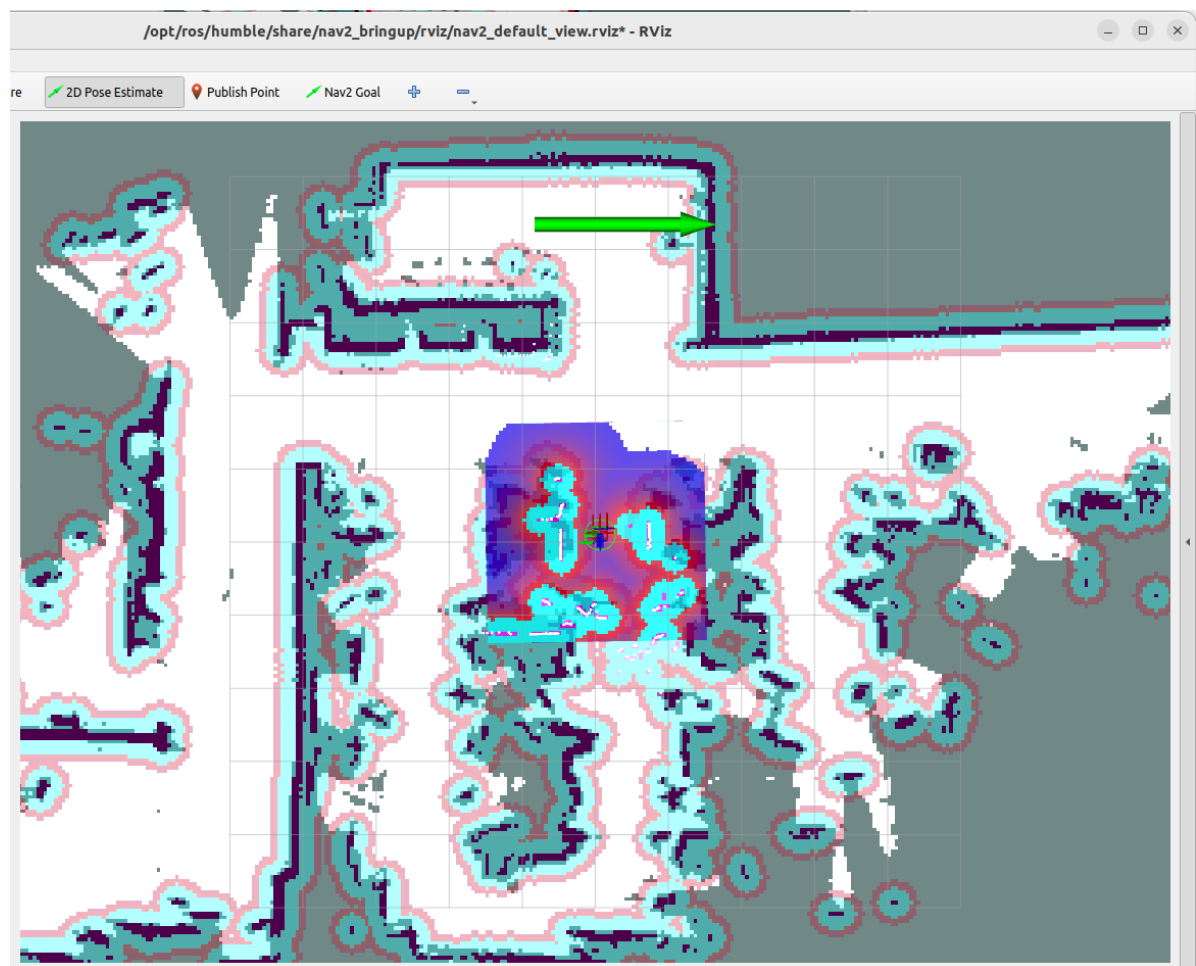
The robot's model will be displayed on the map, as shown below:



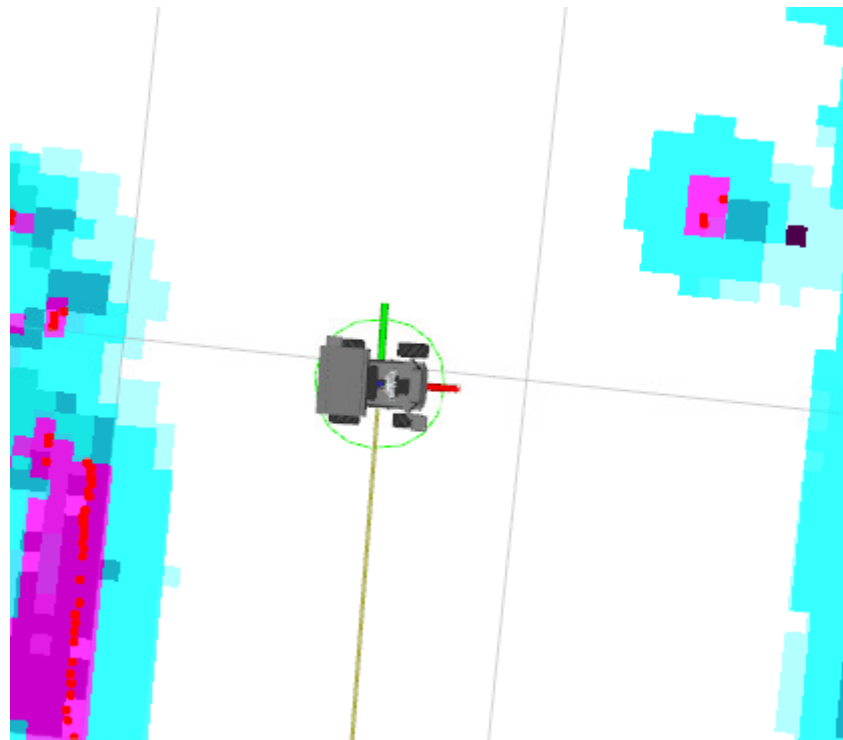
We can name any precise point on the map. Here, we use "Master Bedroom" and "Kitchen" as examples.



As shown in the image below, we first click on **Nav2** The Goal tool guides the vehicle's navigation system to the target location we need to mark.



Wait for the vehicle's navigation system to reach the target location to be marked.



- Run the following command in the terminal to obtain the robot's pose information in the map coordinate system:

```
ros2 run tf2_ros tf2_echo map base_footprint
```

```

jetson@yahboom:~$ ros2 run tf2_ros tf2_echo map base_footprint
[INFO] [1754377006.239762954] [tf2_echo]: Waiting for transform map -> base_footprint: Invalid frame ID "map" passed to canTransform argument target_frame - frame does not exist
[INFO] [1754377008.204300151] [tf2_echo]: Waiting for transform map -> base_footprint: Invalid frame ID "map" passed to canTransform argument target_frame - frame does not exist
At time 1754377008.700000000
- Translation: [3.974, -2.634, 0.000]
- Rotation: in Quaternion [0.000, 0.000, -0.688, 0.726]
- Rotation: in RPY (radian) [0.000, 0.000, -1.516]
- Rotation: in RPY (degree) [0.000, 0.000, -86.867]
- Matrix:
  0.055  0.999  0.000  3.974
 -0.999  0.055 -0.000 -2.634
 -0.000  0.000  1.000  0.000
  0.000  0.000  0.000  1.000
At time 1754377009.734000000
- Translation: [3.974, -2.634, 0.000]
- Rotation: in Quaternion [0.000, 0.000, -0.688, 0.726]
- Rotation: in RPY (radian) [0.000, 0.000, -1.517]
- Rotation: in RPY (degree) [0.000, 0.000, -86.943]
- Matrix:
  0.053  0.999  0.000  3.974
 -0.999  0.053 -0.000 -2.634
 -0.000  0.000  1.000  0.000

```

Open the `map_mapping.yaml` map mapping file (you can choose any method to open it, such as VNC, vscode, or command line):

- Here's an example of opening the file via command line:

```
nano ~/yahboomM3_ws/user_ws/src/m3_bringup/config/multi_brains/map_mapping.yaml
```

```

GNU nano 6.2 /home/jetson/M3Pro_ws/multi_brains_file/map_mapping.yaml
#根据实际的场景环境，自定义地图中的区域，可以添加任意个区域，注意和大模型的地图映射保持一致即可
#According to the actual scene environment, customize the areas in the map. You can add any number of areas, just make sure
#地图映射Map mapping

common_map_areas: #常规导航 common navigation
A:
  name: 'xxx'
  position:
    x: 1.824
    y: -0.838
    z: 0.0
  orientation:
    x: 0.0
    y: 0.0
    z: -0.663
    w: 0.748
B:
  name: 'xxx'
  position:
    x: 1.488
    y: 0.661
    z: 0.0
  orientation:
    x: 0.0
    y: 0.0
    z: 0.725
    w: 0.688

```

- Modify the symbolic pose under the `common_map_areas` field, where `name` is the location name, and fill in the previously obtained poses into the `position` and `orientation` fields.

```

#根据实际的场景环境，自定义地图中的区域，可以添加任意个区域，注意和大模型的地图映射保持一致即可
#According to the actual scene environment, customize the areas in the map. You can add any number of areas, just make sure they are consistent with the map mapping of the large model
#地图映射Map mapping

```

```

common_map_areas: #常规导航 common navigation
A:
  name: 'Master bedroom'
  position:
    x: 3.974
    y: -2.634
  orientation:
    x: 0.0
    y: 0.0
    z: -0.688

```

```

w: 0.726
B:
  name: 'xxx'
  position:
    x: 1.488
    y: 0.661
    z: 0.0
  orientation:
    x: 0.0
    y: 0.0
    z: 0.725
    w: 0.688

```

- Save the file after making changes for the changes to take effect immediately.

2.2 Configuring Map Mapping Variables in Dify

- After configuring the map mapping files mentioned above, we also need to ensure that the AI model knows the relationships between locations and symbols in these maps in advance.
- Start the Dify service (if it's already running, you don't need to start it again):

```
bringup_dify
```

```

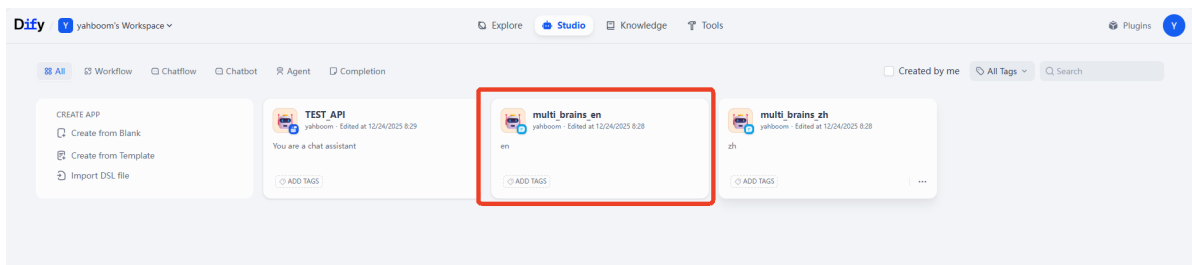
pi@raspberrypi: ~
pi@raspberrypi: ~ 80x24
pi@raspberrypi:~ $ bringup_dify
[+] up 14/14
✓ Network docker_default Created 0.0s
✓ Network docker_ssrf_proxy_network Created 0.1s
✓ Container docker-init_permissions-1 Exited 7.8s
✓ Container docker-sandbox-1 Created 2.9s
✓ Container docker-web-1 Created 2.9s
✓ Container docker-redis-1 Created 2.9s
✓ Container docker-weaviate-1 Created 2.8s
✓ Container docker_ssrf_proxy-1 Created 2.9s
✓ Container docker-db_postgres-1 Healthy 7.8s
✓ Container docker-worker-1 Created 0.3s
✓ Container docker-api-1 Created 0.3s
✓ Container docker-worker_beat-1 Created 0.3s
✓ Container docker-plugin_daemon-1 Created 0.2s
✓ Container docker-nginx-1 Created 0.3s
pi@raspberrypi:~ $

```

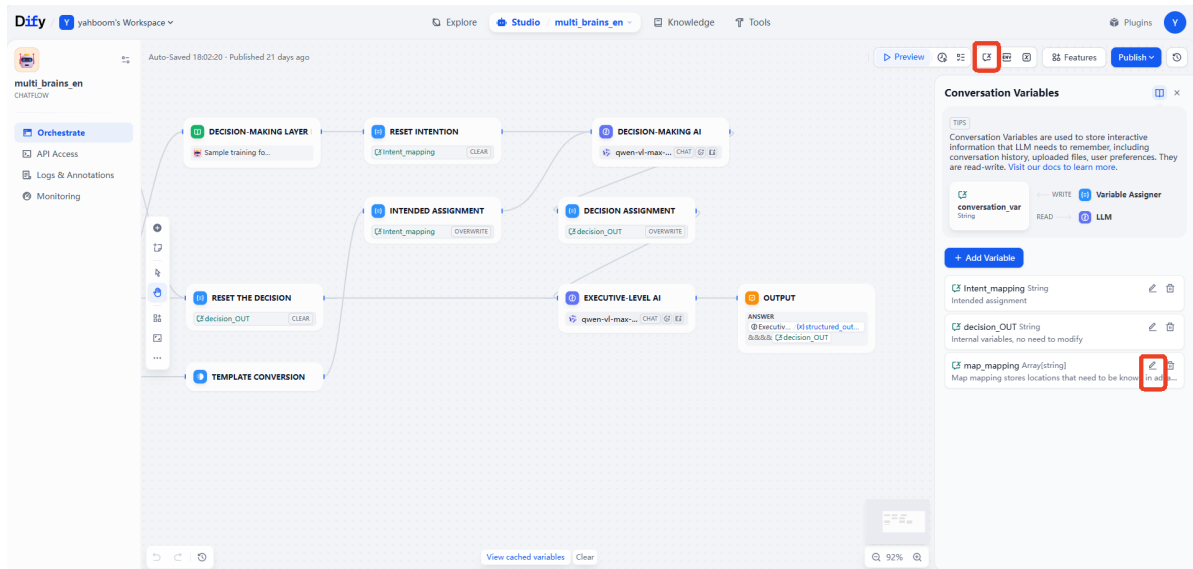
- Enter the vehicle's IP address directly into your browser's address bar to access the Dify management page, then click to select the corresponding AI application.

[!NOTE]

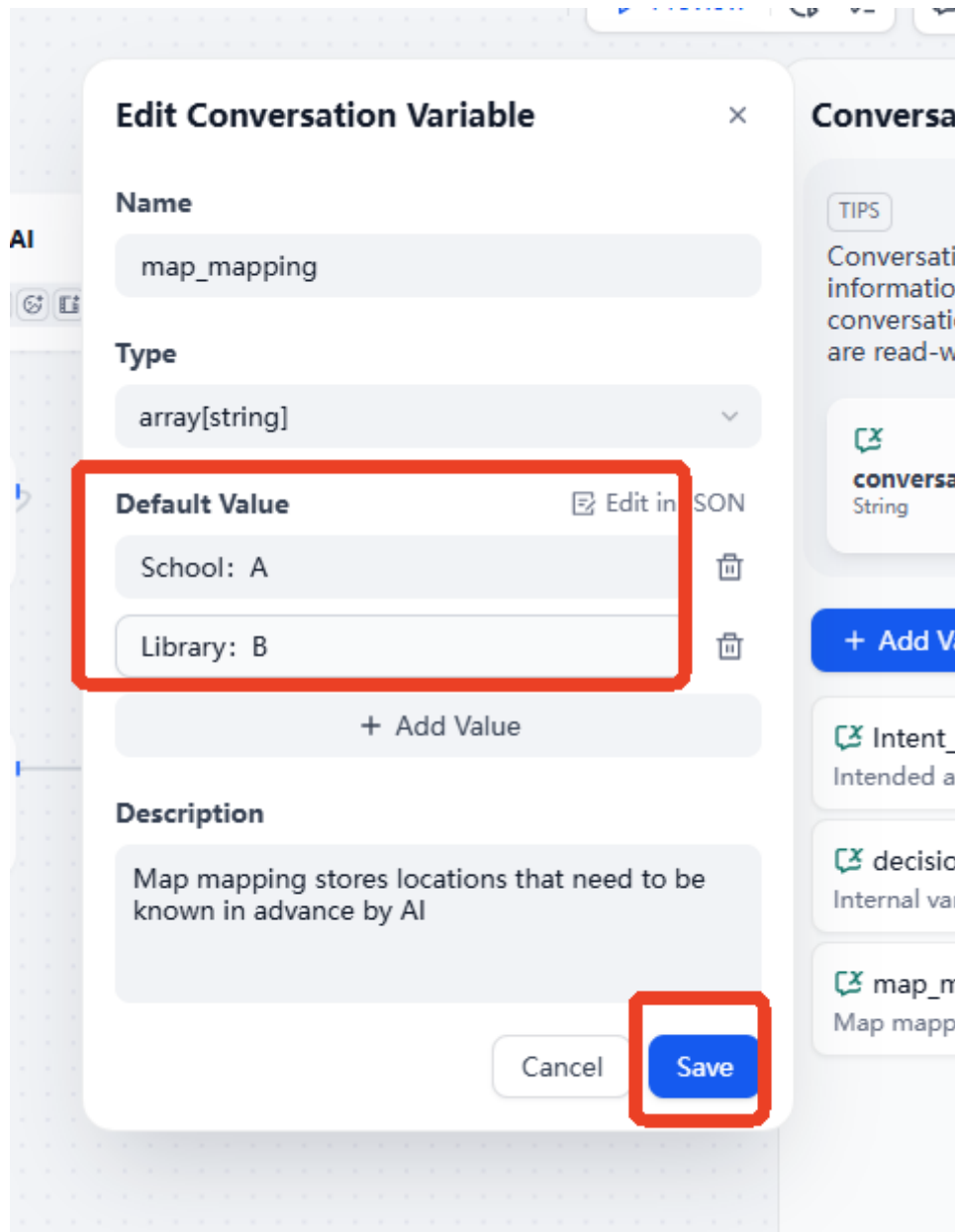
- Domestic users: multi_brains_zh
- International users: multi_brains_en



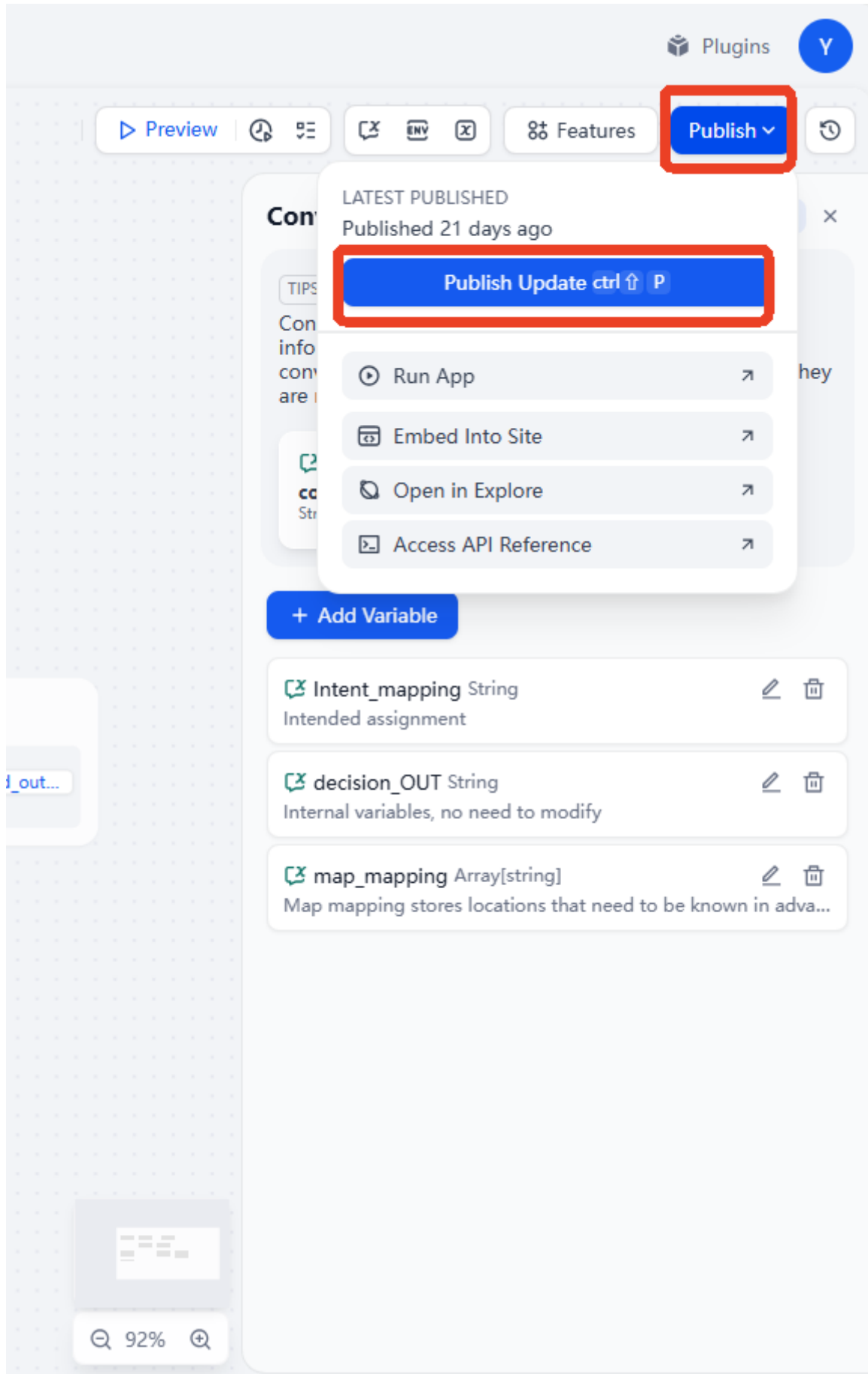
- Click to select **Session variables** in the upper right corner, then click the edit button for the **map_mapping** variable.



- In the pop-up **Edit session variables** window, edit the mapping relationship according to the settings in the previous map mapping file, then click Save.



- Finally, remember to click Publish -> Publish Update to save the changes.



3. Running the Case

3.1 Startup Program

For Raspberry Pi 5 users, if the ROS container is not running, you need to start it first.

```
bringup_m3
```

Open the terminal inside the container.

```
exec_m3
```

VNC connect to the robot desktop, open a terminal, and run the startup command:

```
ros2 launch m3_bringup llm_agent_control.launch.py text_chat_mode:=True
```

- Navigation startup command on the vehicle's infotainment system:

(ORIN & PI5 mainboard)

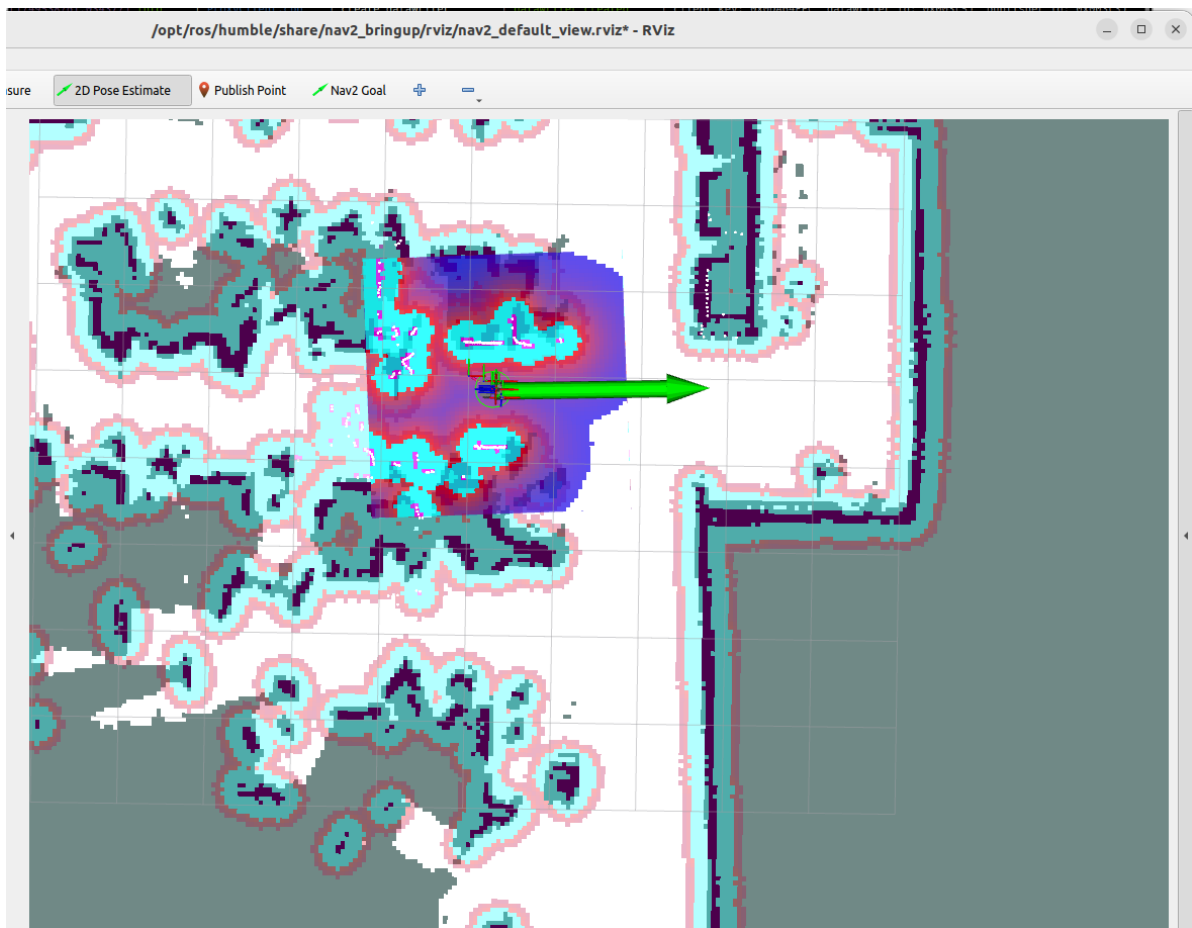
```
ros2 launch m3_bringup nav2.launch.py
```

(RDK mainboard) Step-by-step execution, visualization on the accompanying virtual machine

Rosmaster-M3-Ubuntu22.04 :

```
#Virtual machine side`  
rviz2 -d /home/yahboom/yahboomM3_ws/user_ws/src/m3_bringup/rviz/nav2.rviz  
# Host side  
ros2 launch m3_bringup nav2.launch.py gui:=False
```

Then, follow the navigation startup procedure to initialize positioning. This will open the rviz2 visualization interface. Click **2D Pose Estimate** in the top toolbar to enter selection mode. Roughly mark the robot's location and orientation on the map. After initializing positioning, the preparation is complete.



- Launch the text-based interactive program in the terminal:

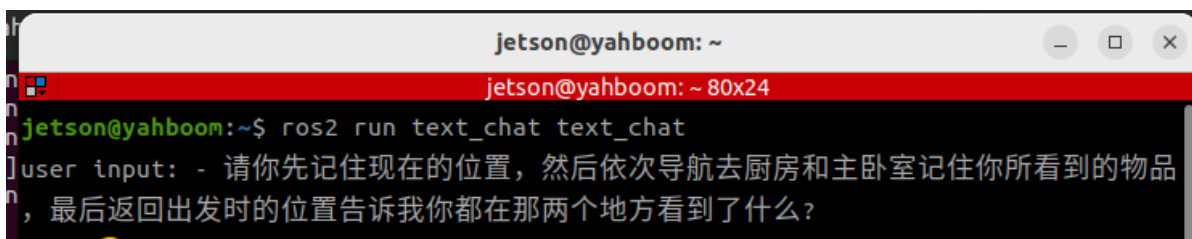
```
ros2 run m3_common_demo text_chat
```

3.2 Test Case

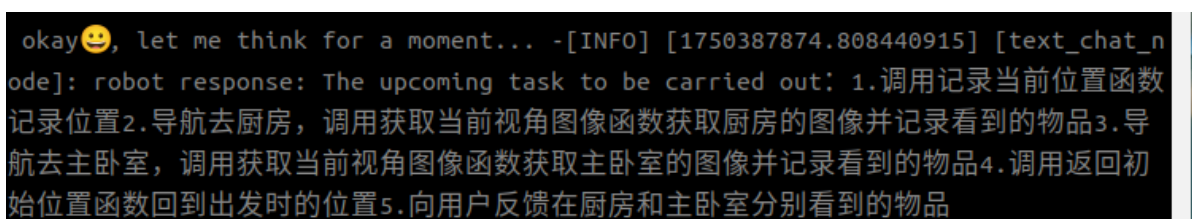
Here is a sample test case. Users can create their own dialogue commands.

- Please remember your current location, then navigate to the kitchen and the master bedroom in that order, remembering the items you see in each room. Finally, return to your starting position and tell me what you saw in those two places.

Copy and enter the above test case into the text-based interactive terminal:



The decision-making AI outputs the planned task steps:



The execution layer AI will then execute the task according to these steps:

user input: [INFO] [1750387878.350106583] [text_chat_node]: robot response: "action": ['get_current_pose()'], "response": 好的，我现在就记住现在的位置，就像记住了一个重要的宝藏地点一样。

[INFO] [1750387882.255020679] [text_chat_node]: robot response: "action": ['navigation(G)'], "response": 我已经记住了当前位置，现在准备导航去厨房啦，就像一个勇敢的探险家一样！

[INFO] [1750387948.665910436] [text_chat_node]: robot response: "action": ['navigation(G)'], "response": 哎呀，刚才去厨房的路上遇到了点小麻烦，不过别担心，我再试一次！

[INFO] [1750387957.637532129] [text_chat_node]: robot response: "action": ['seewh at()'], "response": 我已经成功到达厨房啦！现在我来观察一下这里都有些什么物品。

[INFO] [1750387963.313431519] [text_chat_node]: robot response: "action": [], "response": 在厨房里，我看到的是光滑的地板和一条绿色的线条，不过没有看到具体的物品呢。可能是视角问题，我们继续前进吧！现在我要去主卧室看看了。

[INFO] [1750387966.771073792] [text_chat_node]: robot response: "action": ['navigation(A)'], "response": 我已经告诉了你在厨房看到的情况，现在我将导航去主卧室，看看那里有什么有趣的发现！

[INFO] [1750388031.398516666] [text_chat_node]: robot response: "action": ['navigation(A)'], "response": 哎呀，去主卧室的路上似乎遇到了一点小问题，不过别担心，我再试一次！

[INFO] [1750388037.382963440] [text_chat_node]: robot response: "action": ['seewh at()'], "response": 我已经成功到达主卧室啦！现在我来观察一下这里都有些什么物品。

[INFO] [1750388045.402025419] [text_chat_node]: robot response: "action": [], "response": 在主卧室里，我看到的同样是光滑的地板，地板上有天然的纹理和一些反光的灯光。这里也没有看到具体的物品呢。可能是视角问题或者是房间比较空旷。现在，我将返回出发时的位置，并告诉你我在厨房和主卧室分别看到了什么。

[INFO] [1750388050.039975630] [text_chat_node]: robot response: "action": ['navigation(zero)'], "response": 我已经告诉了你在主卧室看到的情况，现在我将返回出发时的位置。

[INFO] [1750388091.405150056] [text_chat_node]: robot response: "action": [], "response": 我已经回到了出发时的位置。总结一下我的发现：在厨房，我看到的是光滑的地板和一条绿色的线条；而在主卧室，我看到的同样是光滑的地板，地板上有天然的纹理和一些反光的灯光。两个地方都没有看到具体的物品，可能是视角问题或者是房间比较空旷。这就是我的观察结果，希望对你有帮助！

[INFO] [1750388104.844933245] [text_chat_node]: robot response: "action": ['finis htask()'], "response": 我已经完成了全部任务，如果你还有其他需要帮忙的，请随时告诉我哦！